

Módulo 4 – Capítulo 05 – Sección 01

Arquitecturas Multiagente

Un agente individual tiene límites naturales de capacidad: su ventana de contexto, el conjunto de herramientas a las que tiene acceso, el dominio de conocimiento que puede manejar simultáneamente, y el tiempo máximo que puede dedicar a una tarea antes de que el costo de las iteraciones se vuelva prohibitivo. Cuando el problema a resolver supera cualquiera de esos límites, la respuesta arquitectónica es la distribución: asignar distintas partes del problema a agentes especializados que colaboran entre sí. Esta es la premisa de las arquitecturas multiagente.

La distinción fundamental con respecto al Capítulo 04 es de coordinación. En el patrón Supervisor-Workers del capítulo anterior, el supervisor es un agente que coordina herramientas a través de sub-agentes. En un sistema multiagente de producción, la coordinación ocurre entre agentes que son entidades autónomas con sus propios estados, modelos, herramientas y ciclos de ejecución. La interfaz entre ellos no es una llamada a función sino un protocolo de comunicación, con todas las implicaciones que eso conlleva: latencia de red, formatos de mensaje, gestión de versiones de protocolo y coordinación de tiempo.

La premisa de las arquitecturas multiagente — "dividamos el problema entre varios agentes especializados" — es seductora pero requiere validación cuidadosa. Dividir un problema entre agentes introduce complejidad de coordinación, posibilidad de conflictos entre outputs, dificultad de depuración distribuida y multiplicación de los costos de inferencia. Estas fricciones solo se justifican cuando las ventajas de la especialización superan los costos de la coordinación: cuando el problema es genuinamente paralelo, cuando cada sub-dominio requiere un conjunto de herramientas o un modelo distinto, o cuando la escala de procesamiento supera la capacidad de un agente individual.

Los cinco principios que estructuran el diseño de sistemas multiagente son:

- **Especialización clara de responsabilidades:** cada agente debe tener un dominio bien definido del que es responsable. Los dominios deben ser distintos y con fronteras

claras. La ambigüedad sobre qué agente es responsable de qué funcionalidad es la fuente más común de duplicación de esfuerzo y conflictos entre outputs.

- **Protocolos de coordinación explícitos:** la comunicación entre agentes debe usar interfaces bien definidas. No hay "comunicación implícita" entre agentes: cada mensaje debe tener un formato, un origen, un destino y una semántica acordados.
- **Intercambio de contexto controlado:** el contexto que un agente comparte con otro debe ser el mínimo suficiente para que el receptor cumpla su función. Compartir demasiado contexto crea dependencias innecesarias; compartir demasiado poco obliga al receptor a inferir información que debería recibir explícitamente.
- **Escalabilidad diferenciada:** los agentes con mayor carga de trabajo deben poder escalarse independientemente de los que tienen menor carga. Si el sistema no permite ese escalado diferencial, la ventaja de la distribución se pierde.
- **Observabilidad distribuida:** cuando el sistema falla, el arquitecto debe poder rastrear qué agente en qué punto produjo el fallo. La observabilidad en sistemas multiagente requiere trazas correlacionadas: un identificador de correlación que viaja a través de todos los mensajes de una sesión de trabajo y permite reconstruir el flujo completo.

Este capítulo desarrolla los mecanismos concretos de estas arquitecturas: cómo se definen los roles, cómo se coordina la ejecución, cómo se comparte y protege el contexto, y cómo se gestiona el riesgo de la autonomía distribuida. El punto de partida es siempre la misma pregunta del Capítulo 01: ¿esta complejidad se justifica por las necesidades del problema?

Módulo 4 – Capítulo 05 – Sección 02

Roles y Responsabilidades

La diferencia entre un sistema multiagente que funciona y uno que produce outputs inconsistentes o redundantes es frecuentemente la claridad de los roles. Cuando dos agentes tienen dominios solapados, ambos pueden intentar resolver la misma parte del problema con resultados distintos, y el sistema no tiene mecanismos para resolver ese conflicto. Cuando los roles son ambiguos, los agentes toman decisiones sobre qué cubrir basándose en su propio razonamiento, lo que produce comportamientos no deterministas. Definir roles con precisión quirúrgica es el primer trabajo del diseñador de sistemas multiagente.

Un sistema multiagente bien diseñado tiene cinco categorías de roles que corresponden a las responsabilidades fundamentales de cualquier proceso complejo:

Agente planificador: responsable de interpretar el objetivo general y descomponerlo en subtareables asignables a otros agentes. El planificador no ejecuta herramientas directamente (salvo herramientas de estado y coordinación). Su función es razonar sobre la estructura del problema, generar un plan, asignar subtareables y monitorear el progreso. En sistemas donde el plan puede cambiar dinámicamente (porque una subtarea falla o produce información que modifica la estrategia), el planificador debe ser capaz de replanificar sobre la marcha.

Agente recuperador (retriever): especializado en obtener información de fuentes de conocimiento — bases vectoriales, APIs de búsqueda, bases de datos estructuradas, sistemas internos. Su salida es siempre información recuperada con metadatos de fuente. Este rol puede descomponerse en sub-agentes especializados por fuente (un agente para documentos internos, otro para búsqueda web, otro para bases de datos) cuando el sistema maneja múltiples tipos de fuentes con características de acceso distintas.

Agente ejecutor: responsable de realizar acciones en sistemas externos — llamadas a APIs, ejecución de código, escritura en bases de datos, envío de comunicaciones. El ejecutor opera sobre el mundo real y sus acciones pueden ser irreversibles. Por esta razón, el ejecutor debe implementar el principio de mínimo privilegio: solo tiene acceso a las herramientas y sistemas

estrictamente necesarios para su función, y documenta cada acción ejecutada con suficiente detalle para ser auditada.

Agente validador / crítico: revisa los outputs de otros agentes antes de que sean utilizados como entradas para pasos posteriores o entregados al usuario final. El validador puede implementar el patrón Reflection del Capítulo 04 a nivel de sistema: cualquier output que no supere las verificaciones de calidad del validador es devuelto al agente productor para corrección. Este rol es especialmente valioso en sistemas donde la calidad es crítica: generación de código, redacción de documentos formales, o recomendaciones con consecuencias de negocio significativas.

Agente sintetizador: integra los outputs de múltiples agentes en un resultado cohesivo. Cuando el planificador ha distribuido la investigación de un tema entre tres agentes recuperadores especializados, el sintetizador toma sus outputs parciales y genera un resultado integrado, coherente y sin duplicaciones. El sintetizador es el último agente en la cadena antes de la entrega al usuario.

La asignación de roles debe documentarse en el diseño de la arquitectura con la misma precisión que los contratos de API: qué recibe cada agente como entrada, qué produce como salida, qué herramientas puede usar, y qué ocurre si falla. Esta documentación es la base para el testing del sistema multiagente — cada agente puede ser probado de forma aislada con entradas sintéticas — y para la depuración cuando el sistema produce resultados incorrectos.

Nota del Arquitecto: *La tendencia natural al diseñar sistemas multiagente es crear demasiados agentes especializados. He visto diseños con doce agentes distintos para tareas que podrían haberse resuelto con tres. Cada agente adicional añade latencia (tiempo de coordinación entre agentes), costo (más llamadas al LLM) y complejidad de depuración (más componentes que pueden fallar). La regla práctica que uso es: si no puedes describir en una sola oración la responsabilidad única de un agente, ese agente probablemente debería fusionarse con otro. La especialización correcta es la que separa responsabilidades que tienen diferentes modelos, herramientas o frecuencias de actualización.*

La definición precisa de roles es el fundamento sobre el que se construyen los mecanismos de coordinación que se explorarán en la siguiente sección. Sin roles bien definidos, los protocolos de coordinación resuelven el problema equivocado.

Módulo 4 – Capítulo 05 – Sección 03

Coordinación entre Agentes

La coordinación es el problema técnicamente más complejo de los sistemas multiagente. Cuando un agente necesita el resultado de otro para continuar su trabajo, debe existir un mecanismo que transmita ese resultado de forma confiable, en el formato correcto, en el momento adecuado. Cuando dos agentes producen outputs sobre el mismo aspecto de un problema y esos outputs son contradictorios, debe existir un mecanismo de resolución. Cuando el sistema debe decidir si una tarea requiere colaboración entre agentes o puede ser resuelta por uno solo, debe existir un mecanismo de enrutamiento. Cada uno de estos problemas tiene soluciones arquitectónicas específicas.

El **Model Context Protocol (MCP)** es el estándar emergente más relevante para la coordinación de agentes en el ecosistema de herramientas de IA. Desarrollado por Anthropic y adoptado por un número creciente de proveedores, MCP define un protocolo de comunicación que permite a los agentes compartir herramientas, recursos y contexto de forma estandarizada. El lector que haya estudiado el Capítulo 07 del Módulo 3 conoce MCP como mecanismo para conectar un LLM con fuentes de datos externas. En el contexto de sistemas multiagente, MCP actúa como el backbone de comunicación: cada agente expone sus capacidades como un servidor MCP, y otros agentes pueden invocar esas capacidades como clientes. Esto permite construir ecosistemas de agentes interoperables donde agentes desarrollados con distintos frameworks o desplegados en distintos entornos pueden colaborar.

El **Agent-to-Agent (A2A) Protocol** de Google es otro estándar de coordinación multiagente, diseñado para la interoperabilidad entre agentes de distintos proveedores y frameworks. A2A define cómo un agente puede descubrir las capacidades de otro, cómo estructurar las solicitudes de delegación, y cómo gestionar respuestas asíncronas cuando la tarea delegada puede tomar tiempo significativo. La distinción con MCP es de nivel: MCP se enfoca en la exposición de herramientas y recursos, mientras que A2A se enfoca en la delegación de tareas completas entre agentes como entidades de primera clase.

La **arquitectura Blackboard** es un patrón de coordinación donde los agentes no se comunican directamente entre sí, sino a través de una pizarra compartida — una estructura de datos persistente y accesible para todos los agentes del sistema. Cada agente puede leer el estado actual de la pizarra, contribuir con su trabajo parcial, y detectar cuando hay nueva información que activa su función. Este patrón es especialmente valioso para sistemas donde el orden de contribución de los agentes no puede determinarse de antemano, o donde múltiples agentes pueden contribuir al mismo aspecto del problema. La pizarra desacopla a los agentes temporalmente: el productor no espera al consumidor. Su principal desafío es la gestión de la consistencia concurrente cuando múltiples agentes escriben simultáneamente.

Los **mecanismos de consenso** son necesarios cuando múltiples agentes producen evaluaciones o recomendaciones sobre el mismo problema y esos outputs pueden ser contradictorios. Un sistema de investigación donde tres agentes buscan en distintas fuentes de información y llegan a conclusiones distintas necesita una estrategia de resolución. Las opciones incluyen: voting (la conclusión con más respaldo entre los agentes es seleccionada), delegación al planificador (el agente planificador recibe todas las conclusiones y decide cuál es más fiable), o Reflection iterativa (los agentes reciben los outputs de los demás y pueden actualizar su posición en rondas sucesivas hasta alcanzar convergencia).

La topología de coordinación del sistema también es una decisión de diseño con implicaciones en resiliencia y escalado:

- **Topología hub-and-spoke:** un agente orquestador central coordina a todos los demás. Es la más fácil de razonar y depurar, pero el orquestador se convierte en punto único de fallo y cuello de botella.
- **Topología peer-to-peer:** los agentes se comunican directamente entre sí según sus roles. Más resiliente, pero más difícil de observar y depurar porque las interacciones no pasan por un punto central.
- **Topología dirigida por eventos:** los agentes publican eventos en un bus de mensajes (Kafka, RabbitMQ, Pub/Sub de GCP) y se suscriben a los eventos relevantes para su función. El desacoplamiento es máximo, la resiliencia es alta, pero la latencia de coordinación puede ser mayor y la trazabilidad requiere correlación de eventos.

Nota del Arquitecto: La coordinación bien diseñada es la que el usuario no ve. Un sistema multiagente que coordina correctamente produce outputs en tiempos razonables, sin errores de comunicación y con trazas que permiten reconstruir qué ocurrió. Un sistema mal coordinado produce timeouts, mensajes perdidos, outputs

inconsistentes y una traza de depuración imposible de seguir. Mi recomendación para equipos que construyen su primer sistema multiagente es comenzar con topología hub-and-spoke con un orquestador simple: es más fácil hacer funcionar un sistema así y migrarlo a topologías más distribuidas una vez que el dominio está bien comprendido.

La coordinación eficiente entre agentes convierte un conjunto de agentes especializados en un sistema capaz de resolver problemas que ninguno podría resolver individualmente. La siguiente sección explora cómo ese sistema gestiona el conocimiento que cada agente necesita para hacer su trabajo.

Módulo 4 – Capítulo 05 – Sección 04

Memoria Compartida

En un sistema multiagente, el conocimiento no pertenece a un solo agente: pertenece al sistema. El agente recuperador produce información que el sintetizador necesita. El planificador actualiza el estado de la tarea que el ejecutor debe consultar antes de actuar. El validador genera feedback que el agente productor debe incorporar en la siguiente iteración. Sin una capa de memoria compartida bien diseñada, cada una de estas transferencias requiere una comunicación explícita punto a punto, lo que aumenta el acoplamiento entre agentes y hace el sistema frágil ante cualquier cambio de composición.

La memoria compartida en sistemas multiagente tiene tres formas según sus características de acceso y consistencia:

Memoria de trabajo compartida (working memory): es el estado de la tarea en curso, accesible y modificable por todos los agentes participantes en la sesión. Contiene el objetivo, el plan, el estado de cada subtarea, los outputs parciales completados y cualquier variable de coordinación. Esta memoria es efímera (desaparece cuando la tarea termina) y requiere consistencia fuerte: si el planificador actualiza el estado de una subtarea como "completada", el ejecutor no debe intentar ejecutarla nuevamente. Redis con operaciones atómicas o una base de datos con control de concurrencia son las opciones más comunes. El acceso debe estar mediado por un protocolo de escritura que evite conflictos: o bien cada agente tiene una sección propia de la memoria de trabajo que solo él puede escribir, o bien todas las escrituras pasan por el orquestador.

Memoria de conocimiento compartida (knowledge memory): es el repositorio de información recuperada y generada durante la tarea, que los agentes pueden consultar pero no modificar arbitrariamente. Cuando el agente recuperador extrae información de múltiples fuentes, la deposita en esta capa para que el sintetizador pueda acceder a ella sin necesidad de repetir las búsquedas. Una base vectorial es la tecnología más adecuada para este nivel: permite búsqueda semántica sobre el conocimiento acumulado, de forma que cada agente puede consultar lo que otros han encontrado sin conocer de antemano qué consultas

realizaron. Weaviate y Qdrant soportan escrituras concurrentes desde múltiples agentes con aislamiento a nivel de colección.

Memoria de largo plazo del sistema (system long-term memory): es el conocimiento persistente entre sesiones que el sistema multiagente ha acumulado. No es el resultado de una tarea específica sino el aprendizaje institucional del sistema: patrones de consultas frecuentes, estrategias que han funcionado en problemas similares, información sobre usuarios y entidades que reaparecen. Esta memoria permite que el sistema mejore con el tiempo sin reentrenamiento: el siguiente equipo de agentes que aborde un problema similar parte de un contexto más rico que el equipo anterior. Su diseño requiere políticas explícitas de retención (qué se conserva, cuánto tiempo), actualización (qué proceso verifica que el conocimiento almacenado sigue siendo válido) y privacidad (qué información de sesiones anteriores puede usarse en sesiones futuras).

El aislamiento es el desafío central de la memoria compartida en sistemas multiagente multi-tenant. Si el sistema atiende a múltiples organizaciones o usuarios simultáneamente, la memoria compartida de una sesión no debe ser accesible desde otra sesión, y los vectores de una organización no deben aparecer en las búsquedas de otra organización. Implementar el aislamiento requiere una estrategia de namespacing o sharding de la memoria compartida por tenant, con verificación de autorización en cada operación de lectura y escritura.

La consistencia eventual es una característica que los sistemas multiagente deben poder tolerar. Cuando el agente A escribe en la memoria compartida y el agente B intenta leerla milisegundos después en un sistema distribuido, B puede no ver aún la escritura de A. Diseñar el sistema para tolerar esta ventana de inconsistencia — por ejemplo, haciendo que el agente B espere confirmación explícita del orquestador antes de proceder, en lugar de asumir que la escritura de A ya es visible — es más robusto que asumir consistencia fuerte y encontrar los bugs de concurrencia en producción.

La memoria compartida bien diseñada es lo que convierte a un conjunto de agentes independientes en un sistema colaborativo real. Sin ella, los agentes trabajan en paralelo pero no juntos. Con ella, el conocimiento fluye entre agentes de forma controlada, el estado de la tarea es transparente para todos los participantes, y el sistema puede recuperarse de fallos parciales porque el estado no vive solo en la memoria del agente que falló.

Módulo 4 – Capítulo 05 – Sección 05

Riesgos y Gobernanza

La autonomía de un sistema multiagente introduce una categoría de riesgo que no existe en sistemas de IA más simples: el riesgo de acciones en cadena. En un chatbot, cada respuesta es generada y evaluada en forma aislada. En un sistema multiagente, el output de un agente se convierte en el input del siguiente, y un error en una etapa temprana puede propagarse y amplificarse a través de la cadena hasta producir consecuencias significativas antes de que ningún humano tenga oportunidad de intervenir. La gobernanza de sistemas multiagente es la disciplina que establece los controles que previenen esas cadenas de error.

El primer control es la definición de límites de acción. Cada agente del sistema debe tener un alcance de acción explícito y verificable: qué herramientas puede invocar, qué sistemas externos puede modificar, qué valores máximos tienen sus parámetros de acción (un agente de compras no puede procesar pedidos superiores a X euros sin aprobación humana). Estos límites no deben ser instrucciones en el prompt del agente — las instrucciones de prompt pueden ser ignoradas o mal interpretadas — sino controles en la capa de herramientas: la herramienta de procesamiento de pedidos simplemente rechaza cualquier solicitud que supere el umbral configurado, independientemente de lo que el modelo solicite.

El segundo control es la auditoría completa de acciones. En un sistema multiagente, cada acción que cualquier agente realiza — cada herramienta invocada, cada mensaje enviado a otro agente, cada estado escrito en la memoria compartida — debe ser registrada con su identificador de sesión, el agente que la realizó, el timestamp, los parámetros exactos y el resultado. Esta traza de auditoría sirve dos propósitos: la depuración en tiempo real ("¿qué exactamente ocurrió para producir este resultado?") y la auditoría retrospectiva ("¿qué acción específica de qué agente produjo este efecto en el sistema externo?"). Sin esta traza, los sistemas multiagente son cajas negras ante cualquier incidente.

El tercer control es la autorización por nivel de impacto. Las acciones que un agente puede ejecutar deben clasificarse por su nivel de impacto e irreversibilidad. Leer información de una base de datos es una acción de bajo impacto y completamente reversible — no requiere

autorización adicional. Enviar un email a un cliente es de impacto moderado e irreversible — puede requerir una verificación automática de contenido. Realizar una transacción financiera o modificar configuración de producción es de alto impacto e irreversible — requiere aprobación humana explícita antes de ejecutarse. Este sistema de clasificación debe ser diseñado antes del despliegue y verificado por el equipo de seguridad.

El cuarto control es el mecanismo de intervención humana (human-in-the-loop). Un sistema multiagente productivo debe tener un mecanismo claro para pausar la ejecución y solicitar confirmación humana cuando el agente detecta que está en una situación fuera de su competencia, cuando una acción supera su umbral de autorización, o cuando la incertidumbre sobre el objetivo es demasiado alta para continuar sin aclaración. Este mecanismo no debe ser un afterthought: debe diseñarse en el protocolo de coordinación desde el inicio, definiendo qué canal se usa para la solicitud, cuánto tiempo se espera la respuesta, y qué sucede si no llega (pausa, abort, escalada).

Los riesgos específicos que el equipo de gobernanza debe evaluar en un sistema multiagente incluyen:

- **Prompt injection a través de herramientas:** un agente que recupera contenido externo (páginas web, documentos de usuarios) puede recibir instrucciones maliciosas embebidas en ese contenido que modifiquen su comportamiento. Los controles de sanitización de inputs son obligatorios en agentes que procesan contenido no confiable.
- **Amplificación de errores:** un error de razonamiento en el agente planificador produce un plan incorrecto que todos los agentes ejecutores siguen. El validador debe ser independiente del planificador y tener capacidad de rechazar planes que no superen las verificaciones.
- **Costos no controlados:** un agente en un bucle inesperado puede generar decenas de llamadas al LLM y a herramientas externas antes de ser detectado. Los límites de iteraciones y los presupuestos de costo por sesión deben implementarse a nivel de infraestructura.
- **Filtración de información entre tenants:** en sistemas multi-tenant, la memoria compartida y los outputs de agentes deben estar aislados por tenant con verificaciones en cada operación.

La gobernanza de sistemas multiagente no es un conjunto de restricciones que limita la utilidad del sistema. Es el conjunto de controles que hace posible que el sistema opere con autonomía de manera sostenible. Sin gobernanza, la autonomía es un riesgo; con gobernanza, la autonomía es una ventaja competitiva.

Módulo 4 – Capítulo 05 – Sección 06

Resumen

Este capítulo desarrolló las arquitecturas multiagente como la siguiente frontera de complejidad en sistemas de IA: sistemas donde múltiples agentes autónomos colaboran, coordinan su trabajo, comparten contexto y producen juntos resultados que ninguno podría alcanzar individualmente. La promesa es real, pero solo se cumple cuando el diseño es riguroso en cada una de sus dimensiones.

La especialización de roles — planificador, recuperador, ejecutor, validador, sintetizador — es el fundamento de cualquier sistema multiagente efectivo. Sin roles con responsabilidades claras y fronteras precisas, el sistema produce duplicación, conflictos y comportamientos no deterministas. La definición de roles no es una decisión que puede delegarse al razonamiento de los agentes: debe estar en el diseño de la arquitectura, documentada con la misma precisión que los contratos de API.

La coordinación entre agentes requiere protocolos explícitos. El Model Context Protocol (MCP) proporciona un estándar para compartir herramientas y recursos entre agentes. El protocolo A2A de Google define cómo delegar tareas completas entre agentes como entidades de primera clase. La arquitectura Blackboard desacopla temporalmente a los agentes a través de una memoria compartida que actúa como intermediario. Los mecanismos de consenso resuelven conflictos cuando múltiples agentes llegan a conclusiones distintas. La elección del mecanismo de coordinación debe basarse en los requisitos de latencia, resiliencia y observabilidad del sistema específico.

La memoria compartida — de trabajo, de conocimiento y de largo plazo — es la infraestructura que hace fluir el conocimiento entre agentes de manera controlada. Su diseño incluye las decisiones más críticas de aislamiento (multi-tenant), consistencia (fuerte vs. eventual), retención y privacidad. Sin estas decisiones tomadas explícitamente, el sistema acumulará deuda técnica que se manifestará como bugs de concurrencia, filtraciones entre tenants o datos obsoletos que degradan la calidad de las respuestas.

La gobernanza del sistema multiagente — límites de acción, auditoría completa, autorización por nivel de impacto, mecanismos de intervención humana — no es una restricción sobre la autonomía sino la condición necesaria para que esa autonomía sea sostenible en producción. Las organizaciones que despliegan sistemas multiagente sin gobernanza descubren que la primera consecuencia inesperada de la autonomía borra toda la confianza construida con el usuario.

El desafío de las arquitecturas multiagente no es técnico: las herramientas, los protocolos y los frameworks existen y son maduros. El desafío es de diseño: definir con precisión cuándo la complejidad de un sistema multiagente está justificada por el problema que resuelve, y cuándo un agente bien diseñado o un sistema RAG son la solución correcta. Esa evaluación es la marca del AI Architect con experiencia.

Los tres capítulos siguientes del módulo cambian de perspectiva: de la arquitectura de los sistemas de IA a la operación de los sistemas en producción. La observabilidad, la seguridad y la escalabilidad son las disciplinas que determinan si un sistema bien diseñado sobrevive el contacto con la realidad.

"Dividir un problema entre varios agentes no garantiza mejores resultados. La calidad de la arquitectura depende de la correcta definición de responsabilidades y de los mecanismos de colaboración."

— Principio de diseño de sistemas multiagente